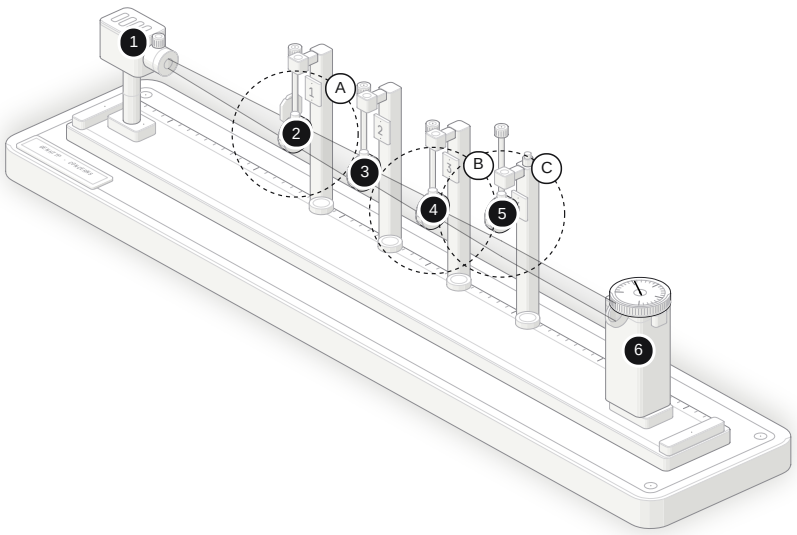
Chain



How does a request pass through auth, CORS and caching without a pipeline framework?

A concern is a handler that owns its inner handler, and Concerns.Chain folds the builders around the tree: the first concern added becomes the outermost ring, so the order in source is the order a request meets. Here auth sits before CORS, compression and caching — close its shutter and nothing downstream sees light at all.

4 rings
AUTH, CORS, COMPRESSION, CACHING ON THIS BENCH

17 shipped
CONCERNS ACROSS THE STANDARD MODULES

1st added
IS THE OUTERMOST — MET FIRST, EVERY TIME

365 days
HSTS PRELOAD DEFAULT ON STRICTTRANSPORT

1 call
HOST.DEFAULTS(...) WIRES THE COMMON RINGS

PARTS

01 **The lamp**
One request, heading in along the rail.

02 **Auth ring**
With a shutter: closed, nothing passes it at all.

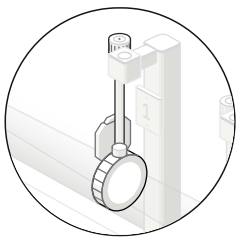
03 **CORS ring**
A slit: per-origin policy on methods, headers, credentials.

04 **Compression ring**
A lens: brotli, gzip or zstd on compressible responses.

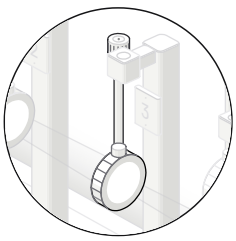
05 **Cache ring**
A mirror: ETag in, If-None-Match answered with a 304.

06 **The meter**
The response — only the light that got through, read here.

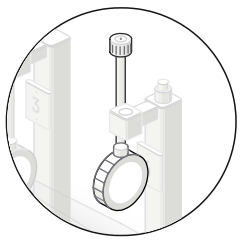
CLOSE-UPS



(A) **The shutter**
Authentication resolves the user into request.SetUser(...) — or blocks. As a ring it guards everything screwed inside it.



(B) **The lens**
CompressedContent rebuilds the response in place: brotli, gzip and zstd, filtered by compressible type, minimum size and Accept-Encoding.



(C) **The mirror**
ClientCache.Validation() generates the ETag and answers If-None-Match with a 304 — the return lamp on the bench.

How it works



01

First added, first met

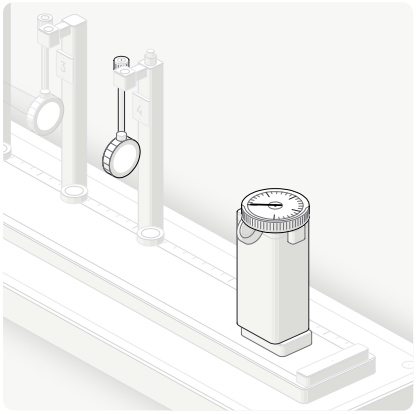
Auth sits outermost, then CORS, then compression: Concerns.Chain folds the builders so source order is encounter order.



02

Close the shutter

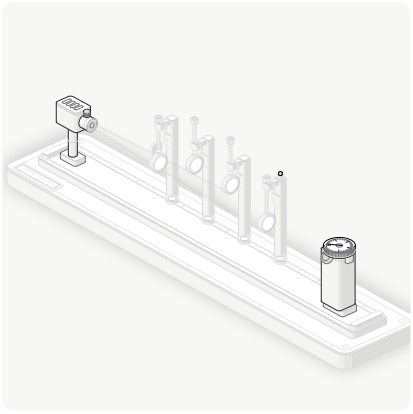
Auth refuses the request and the chain stops there — nothing downstream sees light; the error concern writes the answer.



03

The 304 return

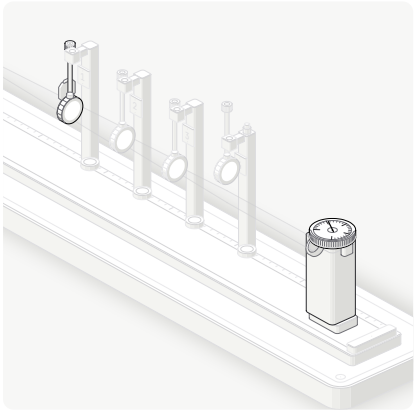
The cache mirror answers from light it already holds: same content, nothing re-sent, one 304 straight back along the rail.



04

Every ring removable

Strip all four and the raw handler still serves, unfiltered. host.Defaults(...) is a convenience, never a requirement.



05

The host builds the same way

Compliance, errors, your concerns, logging: the server's root is one Concerns.Chain too, and .Add(...) scopes another to any subtree.

WHY NOT MIDDLEWARE?

A pipeline framework needs its own interface, its own ordering rules and its own composition root. A concern is only a handler that holds the next handler — so the same two-method contract runs the cross-cutting layers, they compose with every module by construction, and any ring can be lifted off without touching the others.

SPECIFICATION

Interface	IConcern : IHandler, owns Content	Scoped	.Add(...) chains one subtree
Chain	Concerns.Chain(builders, tree)	Defaults	host.Defaults(...) in one call
Order	first added is outermost	Errors	ErrorHandler via IErrorMapper<T>
Root	compliance, errors, concerns, logging		